Excel for Finance

The Complete Professional Handbook

Module 3

Excel Like a Pro

Conditional Formatting, Logical & Lookup Functions, Conditional Aggregation, Error Handling, String Functions, What-If Analysis & Core Financial Functions — with Python and SQL Equivalents

Companion modules in this handbook:

- **Module 1** — Excel Foundations: Interface, Data & Number Formatting, Formula Fundamentals, Shortcuts
- **Module 2** — Making Excel Professional: Optimisation, Macros, Absolute/Relative References, Named Ranges, Data Validation & Custom Formatting
- **Module 3** — Excel Like a Pro: Conditional Formatting, Logical Functions, Conditional Aggregation, Error Handling, Lookup Functions (VLOOKUP/HLOOKUP/XLOOKUP/INDEX-MATCH), String Functions, What-If Analysis, and Core Financial Functions (PMT & Dates) *(this document)*
- **Module 4** — (upcoming) Accounting Fundamentals & Financial Statements, or Charts/PivotTables, depending on which thread you want to pull next

Compiled study notes covering the Excel curriculum end-to-end, extended with equivalent Python (`pandas`, `openpyxl`, `numpy_financial`, `scipy`) and SQL implementations of every concept, for a tech-background learner building cross-tool fluency in financial analytics.

August 15, 2026

Contents

1 How to Use This Module **1**

2 From Formulas to Judgment: What This Module Adds **1**

3 Find and Replace, and Conditional Formatting **2**
3.1 Find and Replace . . . 2
3.2 Conditional Formatting . . . 4
3.3 Python and SQL Equivalents . . . 5

4 Logical Functions: IF, Nested IF, and IFS **6**
4.1 Python and SQL Equivalents . . . 7

5 Conditional Aggregation Functions **8**
5.1 Criteria Syntax — The Part That Trips People Up . . . 8
5.2 COUNTIF / COUNTIFS . . . 9
5.3 AVERAGEIF / AVERAGEIFS . . . 9
5.4 SUMIF / SUMIFS . . . 9
5.5 MAXIFS / MINIFS . . . 10
5.6 Python and SQL Equivalents . . . 10

6 Error Handling **11**
6.1 Reading Excel's Built-In Error Codes . . . 11
6.2 IFERROR and IFNA . . . 12
6.3 IS-Family Testing Functions . . . 13
6.4 Python and SQL Equivalents . . . 13

7 Lookup Functions: VLOOKUP, HLOOKUP, XLOOKUP, and INDEX/MATCH **14**
7.1 VLOOKUP . . . 14
7.2 HLOOKUP . . . 15
7.3 XLOOKUP . . . 15
7.4 INDEX and MATCH . . . 16
7.5 Choosing the Right Lookup Function . . . 17
7.6 Python and SQL Equivalents . . . 17

8 String Functions **18**
8.1 Extracting Substrings by Position . . . 18
8.2 Finding a Position, and Extracting by It . . . 19
8.3 Cleaning Text . . . 19
8.4 Case Conversion . . . 19
8.5 Joining Text . . . 20
8.6 Replacing Text: SUBSTITUTE vs. REPLACE . . . 20
8.7 Number-Text Conversion: TEXT and VALUE . . . 20
8.8 Python and SQL Equivalents . . . 20

9 What-If Analysis **21**
9.1 The Core Idea . . . 21
9.2 Data Tables . . . 22
9.2.1 One-Variable Data Table . . . 22
9.2.2 Two-Variable Data Table . . . 22
9.3 Scenario Manager . . . 23
9.3.1 Analysing Multiple Scenarios at Once: The Scenario Summary Report . . . 24
9.4 Goal Seek . . . 25

9.4.1 Reading the Goal Seek Status Dialog . . . 26
9.5 Choosing the Right What-If Tool . . . 27
9.6 Python and SQL Equivalents . . . 27

10 Core Financial Functions: PMT and Date Functions **28**
10.1 The PMT Function . . . 29
10.2 Date Functions . . . 30
10.3 Python and SQL Equivalents . . . 32

11 Worked Example: A Loan Sensitivity Model, Three Ways **33**
11.1 Excel Implementation . . . 33
11.2 Python Implementation . . . 33
11.3 SQL Implementation . . . 34

12 Common Pitfalls & Debugging Checklist **35**

13 Chapter Summary & Self-Check **36**

14 What's Next: Module 4 Preview **36**

1 How to Use This Module

This module assumes you have completed Modules 1 and 2: you should be comfortable navigating the interface, formatting data, writing basic formulas with relative references, using absolute/mixed references, and building Named Ranges and Data Validation rules. Everything here builds directly on that foundation — in particular, several pitfalls flagged in Module 2 (positional fragility, volatile functions slowing recalculation, the one-time nature of Text to Columns) get their proper fix in this module.

A Note on Scope — Why This Module Is Larger Than "Excel Like a Pro" Alone

The original course structure splits this material across two separate modules: *Excel Be a Pro* (Find/Replace, Conditional Formatting, IF/IFS, the COUNTIF/SUMIF family, error handling, all four lookup functions, and string functions) and, later, *Financial Functions* (What-If Analysis and PMT/date functions). This handbook document merges both into a single Module 3, because What-If Analysis and PMT/date calculations lean directly on the logical functions, lookup functions, and error-handling patterns taught earlier in this same module — splitting them apart would mean re-explaining prerequisites twice. Module 4 will pick up where the original course's Module 7 begins: Accounting Fundamentals and the three financial statements.

As in Modules 1 and 2, every topic is explained three times, in three dialects:

- **Excel** — the native implementation: menu paths, dialog boxes, formula syntax, and keyboard shortcuts, exactly as you would click and type them.
- **Python** — the equivalent using `pandas` (vectorised conditional logic, string methods, merges), `numpy_financial` (time-value-of-money functions, since NumPy itself dropped its financial functions), and occasionally `scipy.optimize` (for Goal-Seek-style backward solving).
- **SQL** — the closest conceptual equivalent: `CASE WHEN` for logical branching, `GROUP BY` for conditional aggregation, and — most importantly in this module — `JOIN` as the real, set-based operation that `VLOOKUP` and `INDEX/MATCH` are Excel's row-by-row imitation of.

The same recurring boxes from Modules 1 and 2 appear throughout: **Key Concept** (blue), **Finance Application** (green), **Pro Tip** (green, lighter), **Common Pitfall** (red), and **Formula Syntax** (white, green border). Where a topic genuinely has no clean equivalent in one of the three tools — and this happens more than once in this module, particularly for What-If Analysis in SQL — that gap is stated plainly rather than papered over with a forced translation.

2 From Formulas to Judgment: What This Module Adds

Modules 1 and 2 were mostly about *mechanics*: getting data into cells correctly, formatting it, and making formulas behave predictably when copied. Nothing so far has made a genuine *decision*. A `SUM` adds; it does not judge. A named range labels a cell; it does not ask a question of the data.

Key Concept

This module is where Excel starts doing the things people actually mean when they say "analysis": branching on a condition (**IF**), aggregating only the rows that meet a rule (**COUNTIF**/**SUMIF**), retrieving a specific fact by looking it up rather than by remembering where it sits (**VLOOKUP**/**INDEX-MATCH**), failing gracefully instead of propagating a broken value (**IFERROR**), reshaping messy text into usable data (string functions), and — in the second half of the module — systematically asking "what happens if an assumption changes?" (**What-If Analysis**) and computing the time-value-of-money mechanics (**PMT** and date functions) that almost every financial model needs on its very first tab.

It is also the module where the case for learning Python and SQL alongside Excel becomes most concrete rather than theoretical. A `VLOOKUP` is a manual, one-column-at-a-time simulation of something a database does natively and efficiently: a `JOIN`. A `SUMIFS` is a manual simulation of a `GROUP BY`. Seeing the Excel version first, and then its native equivalent, is the fastest way to understand *why* those tools exist and what problem they were built to solve — not just how to operate them.

3 Find and Replace, and Conditional Formatting

These two tools share a section because they answer related questions: Find and Replace *locates and changes* specific values; Conditional Formatting *locates and visually flags* values without changing them. Together they are the first line of defence for auditing and presenting a worksheet.

3.1 Find and Replace

Find and Replace — Access

`Ctrl+F` opens **Find**; `Ctrl+H` opens **Replace** (the same dialog, one tab further). Both live under **Home** → **Editing** → **Find & Select** as well.

Clicking **Options »** inside the dialog reveals the settings that separate a casual search from an audit-grade one:

Option	What it controls
Within	**Sheet** (current tab only) vs. **Workbook** (every tab) — easy to forget and search only one tab of a multi-tab model.
Search	**By Rows** vs. **By Columns** — changes the order results are stepped through with **Find Next**, not what is found.
Look in	**Formulas** (default — searches the underlying formula text, e.g. finds a cell containing `=VLOOKUP(...)` if you search `VLOOKUP`), **Values** (searches the *displayed result*, e.g. finds a cell showing `10,623.52` regardless of what formula produced it), or **Comments**.
Match case	Case-sensitive search — off by default.
Match entire cell contents	Only matches a cell whose *entire* content equals the search term, not just a substring — critical when searching for a short code like `"5"` that would otherwise also match `"250"`, `"5000"`, and every other cell containing that digit.
Format…	Restricts the search to cells carrying specific *formatting* (a fill colour, a number format, a font) — independent of the cell's text content.

Wildcards work inside both Find and Replace: `*` matches any run of characters, `?` matches exactly one character. Searching `Inv*2026` finds `Invoice-2026`, `Inv Final 2026`, and anything else starting with `Inv` and ending in `2026`.

Searching for a Literal * or ?

Because `*` and `?` are wildcard operators, searching for an actual asterisk or question mark in your data requires escaping it with a tilde: `~*` finds a literal `*`, `~?` finds a literal `?`. Forgetting this turns a search for a literal character into an unintentional wildcard search that matches almost everything.

Find and Replace as an Audit Tool, Not Just an Editing Tool

Find All (in the Find tab) lists every match in a sortable grid at the bottom of the dialog, with sheet name, cell address, and value — clicking a row jumps straight to that cell. This turns Find and Replace into a fast way to answer audit questions that have nothing to do with editing: "does the hardcoded number `0.21` appear anywhere in this model outside the tax-rate assumption cell?" Searching with **Look in: Formulas** and **Find All** surfaces every hardcoded instance in seconds — exactly the kind of stray hardcoded input that a Named Range (Module 2, § Named Ranges) is meant to prevent, and Find All is how you verify none slipped through.

Finance Application

A common real use: a fiscal year label (`"FY2025"`) is typed as literal text in dozens of report headers across a workbook instead of being built from a `YEAR()` formula. Replace All with **Within: Workbook** updates every tab in one pass when the year rolls over — versus the alternative of manually hunting through every sheet.

3.2 Conditional Formatting

Key Concept

Conditional Formatting (CF) is a *presentation layer*, not a data layer. It changes how a cell *looks* based on a rule, but never changes the cell's actual value — the opposite of Data Validation (Module 2), which restricts what value can be *entered* in the first place. A cell can fail every CF rule in the workbook and still contain a perfectly valid number; CF only affects what the eye notices.

Access via **Home → Conditional Formatting**, with five built-in rule families plus a fully custom option:

Rule type	What it does
Highlight Cells Rules	Greater Than / Less Than / Between / Equal To / Text that Contains / Duplicate Values — simple threshold or text-match flags with a preset colour fill.
Top/Bottom Rules	Top 10 Items, Top 10%, Above Average, Below Average — flags based on the cell's rank or position relative to the rest of the selected range, recalculated live as data changes.
Data Bars	An in-cell horizontal bar whose length is proportional to the value — turns a column of numbers into an instant bar chart without leaving the cell.
Colour Scales	A 2- or 3-colour gradient (e.g. red→yellow→green) mapped across the range's min/mid/max — a heat-map view of magnitude.
Icon Sets	Arrows, traffic lights, or flags assigned by value tier (e.g. green up-arrow / yellow flat-arrow / red down-arrow).
New Rule → **Use a formula**	Fully custom: any formula that evaluates to `TRUE`/`FALSE` for the *active cell* of the selected range triggers the format — the most powerful and most commonly used option in real financial models.

Formula-Based CF — The Reference Convention

When you write a formula-based rule over a selected range (say `C5:C40`), the formula must be written *relative to the top-left, active cell of that selection* (`C5`) — exactly like typing a formula into `C5` and letting it fill down/across. Excel then applies the same relative-shift logic from Module 2 to every other cell in the range automatically. A rule like `=C5<0` applied to `C5:C40` flags every cell in that column that is individually negative; a rule like `=C$4="Over Budget"` (row locked, column free) flags every cell in a range based on a single header-row lookup applying across all columns.

The #1 Cause of "CF Applied to the Wrong Cells"

This relative-to-top-left-cell convention is the single most common source of confusion with formula-based CF: if the range was selected starting from `C10` instead of `C5`, but the formula was written as if `C5` were the anchor, every row's evaluation is silently offset by five rows. Always double-check **Applies to** (visible in **Manage Rules**) matches the range you actually intended, and write the formula referring to the *first* cell of that exact range.

Manage Rules (**Conditional Formatting → Manage Rules**) lists every rule on the sheet (or workbook), in priority order (top rule evaluated first), lets you edit, delete, and reorder them,

and exposes **Stop If True** — a checkbox that, when a higher-priority rule matches, prevents any lower-priority rule from also applying to that same cell (directly analogous to the "first match wins" behaviour of a nested `IF` chain, covered next in §4).

Finance Application

Three CF patterns appear constantly in financial models: (1) a formula rule `=C5<0` with a red font, applied to every variance column, to make negative deviations from budget visually unmissable; (2) a Colour Scale over a correlation matrix or a sensitivity grid (exactly the two-way sensitivity table built with mixed references in Module 2's worked example) to turn a wall of numbers into an at-a-glance heat map; (3) a Data Bar over a single column of revenue-by-product figures, giving stakeholders a bar-chart impression without inserting an actual chart object.

3.3 Python and SQL Equivalents

Python. `openpyxl` can write real Conditional Formatting rules *into* an `.xlsx` file (so the rule remains live and dynamic when the user opens it in Excel); `pandas`'s `DataFrame.style` applies the equivalent visual logic *inside a Python/Jupyter session* for on-screen or exported-HTML review, without writing back to Excel's own rule engine.

Listing 1: Both flavours: a live openpyxl rule for the .xlsx file, and a one-off pandas.style view for analysis

```python
from openpyxl import Workbook
from openpyxl.formatting.rule import CellIsRule, ColorScaleRule
from openpyxl.styles import Font, PatternFill

wb = Workbook(); ws = wb.active
for i, v in enumerate([15000, -4200, 8300, -900], start=2):
    ws.cell(row=i, column=3, value=v)

# Highlight Cells Rules equivalent: red font for negatives
red_font = Font(color="922B21")
ws.conditional_formatting.add(
    "C2:C5", CellIsRule(operator="lessThan", formula=["0"], font=red_font
        )
)
# Colour Scale equivalent
ws.conditional_formatting.add(
    "C2:C5",
    ColorScaleRule(start_type="min", start_color="F8696B",
                   end_type="max", end_color="63BE7B"),
)
wb.save("cf_demo.xlsx")

# --- pandas: on-screen/report equivalent, not written back to Excel's
    rule engine ---
import pandas as pd
df = pd.DataFrame({"variance": [15000, -4200, 8300, -900]})
styled = df.style.applymap(
    lambda v: "color: #922B21; font-weight: bold" if v < 0 else ""
)  # displays inline in Jupyter/exports to styled HTML
```

SQL. There is no visual layer in a database, so the honest equivalent of Conditional Formatting is computing the *flag itself* as a column, using `CASE WHEN` — the same construct that will reappear as the direct SQL equivalent of `IF` in §4. Find and Replace's equivalent is a straightforward `UPDATE ... SET ... WHERE`.

Listing 2: Computing a CF-equivalent flag column, and a Find/Replace-equivalent UPDATE

```
-- Conditional-Formatting equivalent: the "flag" becomes an explicit
    column
SELECT
    account_name,
    variance,
    CASE WHEN variance < 0 THEN 'Over Budget' ELSE 'On Track' END AS
       status_flag
FROM budget_variance;

-- Find and Replace equivalent: bulk-updating stored text
UPDATE report_headers
SET header_text = REPLACE(header_text, 'FY2025', 'FY2026')
WHERE header_text LIKE '%FY2025%';
```

4 Logical Functions: IF, Nested IF, and IFS

IF

`=IF(logical_test, value_if_true, [value_if_false])` — evaluates `logical_test` (any expression that resolves to `TRUE` or `FALSE`), returning the second argument if true and the third if false. The third argument is optional; omitting it returns `FALSE` when the test fails, which is rarely what you want in a finance model, so it is treated as effectively required in practice.

A single `IF` handles a two-way branch. Real decisions usually have more than two outcomes, which historically forced **nested IFs** — an `IF` inside the `value_if_false` of another `IF`, chained as deep as needed:

Listing 3: A three-tier bonus schedule, nested IF style

```
=IF(C5>=1000000, "Platinum", IF(C5>=500000, "Gold", IF(C5>=100000, "
    Silver", "Bronze")))
```

Key Concept

Nested `IF` evaluates **top to bottom, and stops at the first `TRUE`**. This means condition order is not cosmetic — it is logic. If the conditions above were written narrowest-first (`C5>=100000` before `C5>=1000000`), a sales figure of 1,200,000 would incorrectly return `"Silver"`, because the broad `>=100000` test is satisfied first and the formula never reaches the more specific, correct tier below it. This exact bug — a broad condition unintentionally shadowing a narrower, more correct one placed after it — is the single most common nested-IF error, and it produces no error message at all; the formula just silently returns the wrong tier.

IFS (Excel 2016+/365) replaces a nested-IF chain with a flat list of condition/result pairs, evaluated in the same top-to-bottom, first-match-wins order, but without the pyramid of closing parentheses:

IFS

`=IFS(logical_test1, value1, [logical_test2, value2], ...)` — same first-match-wins semantics as nested IF, read as a flat decision table. If no test matches and no catch-all is supplied, `IFS` returns `#N/A` — so a final `TRUE` condition (`TRUE, "Bronze"`) is the conventional way to write a default/else branch.

Listing 4: The same bonus schedule, IFS style

```
=IFS(C5>=1000000,"Platinum", C5>=500000,"Gold", C5>=100000,"Silver", TRUE
    ,"Bronze")
```

	Nested IF	**IFS**
Readability past 3–4 tiers	Degrades fast — a wall of closing parentheses	Stays flat and scannable regardless of tier count
Excel version required	Every version	2016+/365 only — breaks in older files
No-match behaviour	Falls through to the innermost `value_if_false` you wrote	Returns `#N/A` unless a `TRUE` catch-all is added

Compound conditions inside either form use `AND()`, `OR()`, and `NOT()`, each taking any number of TRUE/FALSE arguments:

Listing 5: Compound logical tests

```
=IF(AND(C5>=500000, D5="Approved"), "Disburse", "Hold")
=IF(OR(E5="Late", E5="Defaulted"), "Flag for Review", "Current")
=IF(NOT(F5="Closed"), "Still Open", "Closed")
```

Finance Application

Nested logic drives loan-approval flags (income threshold *and* credit-score threshold *and* debt-to-income below a cap, via `AND`), covenant pass/fail tests in a debt schedule, bonus-tier and commission-tier calculations, and simple three-statement flags like classifying a balance-sheet line as Current vs. Non-Current based on a maturity-date comparison.

Deeply Nested IFs Are a Maintenance Trap, Not Just an Eyesore

Beyond about four or five nested levels, a formula becomes practically impossible to audit by eye, and a single misplaced parenthesis silently breaks the logic rather than throwing a clear error. Excel enforces a hard technical limit of 64 nested `IF` functions, but the *practical* limit — the point past which a human reviewer can no longer verify correctness — is much lower. `IFS`, or a lookup-table approach (VLOOKUP/XLOOKUP against a tier table, §7), is the standard fix once a nested-IF chain exceeds three or four branches.

4.1 Python and SQL Equivalents

Python. A scalar `if/elif/else` chain is the direct one-value-at-a-time equivalent. For an entire column at once, `pandas` offers two idiomatic, vectorised alternatives that map cleanly onto Excel's two tools: `np.where` for a single two-way branch (IF), and `np.select` or `pd.cut` for a multi-tier ladder (nested IF/IFS) — notably *faster* than a Python loop because they operate on the whole column in one vectorised pass rather than row by row.

Listing 6: IF and the tiered bonus schedule, three Python ways

```
import numpy as np
import pandas as pd

df = pd.DataFrame({"sales": [1200000, 620000, 150000, 40000]})

# --- np.where: the direct IF() equivalent ---
```

```
df["disburse"] = np.where(df["sales"] >= 500000, "Disburse", "Hold")

# --- np.select: the nested-IF / IFS equivalent, explicit conditions list ---
conditions = [df["sales"] >= 1_000_000, df["sales"] >= 500_000, df["sales"] >= 100_000]
choices    = ["Platinum", "Gold", "Silver"]
df["tier"] = np.select(conditions, choices, default="Bronze")

# --- pd.cut: purpose-built for tiering a continuous number into bins ---
df["tier_v2"] = pd.cut(
    df["sales"], bins=[-np.inf, 100_000, 500_000, 1_000_000, np.inf],
    labels=["Bronze", "Silver", "Gold", "Platinum"]
)
```

`np.select` Mirrors `IFS` Exactly — Order Still Matters

`np.select` evaluates its `conditions` list top to bottom and, like `IFS`, uses the *first* matching condition for each row — so the same broad-before-narrow ordering bug from Excel can just as easily be written in Python. `pd.cut` sidesteps the ordering question entirely by requiring sorted bin edges, which is why it is often the more robust choice specifically for tiering a continuous numeric variable.

SQL. `CASE WHEN` is SQL's native branching construct, and it is a closer match to `IFS` than to a single `IF` — it was designed from the start as a flat, ordered, multi-branch expression.

Listing 7: The bonus schedule as a SQL CASE expression

```
SELECT
    customer_id,
    sales,
    CASE
        WHEN sales >= 1000000 THEN 'Platinum'
        WHEN sales >= 500000  THEN 'Gold'
        WHEN sales >= 100000  THEN 'Silver'
        ELSE 'Bronze'
    END AS tier
FROM customer_sales;
```

5 Conditional Aggregation Functions

Key Concept

Every function in this family answers the same underlying question: "aggregate this data, but only the rows that meet a rule." The rule — called the **criteria** — is where almost all of the subtlety lives; the aggregation itself (count, sum, average, max, min) is trivial once the criteria syntax is understood.

5.1 Criteria Syntax — The Part That Trips People Up

Criteria can be a literal number, a literal text string (in quotes), a comparison operator as a string, a wildcard pattern, or a reference built with concatenation:

Criteria written as	Matches	Notes
`100`	Exactly 100	Numeric literal, no quotes
`"Approved"`	Exactly the text `Approved`	Text literal, in quotes; not case-sensitive
`">100"`	Any value greater than 100	Operator *and* number together inside one string
`"<>Approved"`	Anything except `Approved`	`<>` is "not equal to"
`"Inv*"`	Any text starting with `Inv`	Wildcard `*`, same rules as Find/Replace (§3.1)
`">="&A1`	Anything ≥ the value in `A1`	Concatenating an operator string with a cell reference — the criteria becomes dynamic instead of hardcoded

5.2 COUNTIF / COUNTIFS

COUNTIF and COUNTIFS

`=COUNTIF(range, criteria)` — counts cells in `range` meeting one criterion. `=COUNTIFS(criteria_range1, criteria1, [criteria_range2, criteria2], ...)` — counts rows meeting *all* criteria simultaneously (an implicit AND across every pair), across up to 127 range/criteria pairs.

5.3 AVERAGEIF / AVERAGEIFS

AVERAGEIF and AVERAGEIFS

`=AVERAGEIF(range, criteria, [average_range])` and `=AVERAGEIFS(average_range, criteria_range1, criteria1, ...)` — same criteria mechanics as COUNTIF/COUNTIFS, but averaging a specified range of values instead of counting matching cells.

AVERAGEIF Silently Ignores Blanks but Not Zeros

Exactly as Module 1 flagged for plain `AVERAGE`: blank cells within the matched criteria are excluded from the average automatically, but cells containing an actual `0` are included and pull the average down. If a "blank" cell is actually a formula returning an empty string `""`, AVERAGEIF treats it as text, not blank, and it is silently skipped in a different way (text is never numeric) — worth double-checking with `COUNTIFS` against the same criteria to confirm the denominator matches expectation.

5.4 SUMIF / SUMIFS

SUMIF

`=SUMIF(range, criteria, [sum_range])` — `range` is checked against `criteria`; `sum_range` (optional — defaults to `range` itself) is what actually gets added for matching rows.

SUMIFS

`=SUMIFS(sum_range, criteria_range1, criteria1, [criteria_range2, criteria2], ...)` — note the **argument order is reversed** relative to SUMIF: the range being summed comes *first* here, not last.

The SUMIF/SUMIFS Argument-Order Trap

This is the single most common typing error in the whole conditional-aggregation family: `SUMIF` places the range to sum *last* (and it's optional), while `SUMIFS` places it *first* (and it's required). Muscle memory built on one function actively produces a broken formula in the other. Excel's Formula AutoComplete tooltip (visible while typing, shown live in the source lecture screenshot for this module) is the fastest way to double-check argument order before hitting Enter, rather than relying on memory.

Mismatched Range Sizes

`criteria_range` and `sum_range` (or `average_range`) must be the *same size and shape* — same number of rows and columns. If `sum_range` is accidentally one row shorter or offset by a row from `criteria_range` (a common result of a careless fill-down or an inserted row that only shifted one of the two ranges), Excel does not raise an error; it silently aligns them by relative position, producing a wrong total with no warning.

5.5 MAXIFS / MINIFS

MAXIFS and MINIFS

`=MAXIFS(max_range, criteria_range1, criteria1, ...)` and the equivalent `MINIFS` — same criteria mechanics again, returning the largest/smallest value among matching rows instead of a count/sum/average. Available from Excel 2019/365 onward; older files need an array formula built from `MAX(IF(...))` entered with `Ctrl+Shift+Enter`, or a helper-column workaround.

Finance Application

`SUMIFS` is the workhorse of departmental expense reporting: rolling up every transaction row in a general-ledger export by cost centre *and* month in a single formula, without a PivotTable. `COUNTIFS` is the natural companion to Module 2's Data-Validation-enforced category lists — counting how many transactions fall in each validated category confirms the dropdown is actually preventing the typo-driven under-counting (`"Marketing"` vs. `"Marketting"`) that free-text entry would allow. `AVERAGEIFS` computes average deal size by region and quarter. `MAXIFS` finds the largest single transaction within a flagged category, useful for spot-checking outliers before they distort a SUMIFS total.

This Whole Family Is Excel Simulating a GROUP BY, One Combination at a Time

Every SUMIFS/COUNTIFS formula answers "for this one specific combination of category and period, what's the total?" A real `GROUP BY` (see the SQL box below) answers that question for *every* combination simultaneously, in one pass, without needing one formula per combination. Keep this framing in mind — it is exactly why a PivotTable (a later module) usually replaces a grid of dozens of individually-typed SUMIFS formulas in practice.

5.6 Python and SQL Equivalents

Listing 8: Boolean-mask aggregation (the direct SUMIF/COUNTIF equivalent) and groupby (the SUMIFS-at-scale equivalent)

```
import pandas as pd

df = pd.DataFrame({
    "cost_center": ["Marketing", "Sales", "Marketing", "Ops", "Sales"],
```

```
    "month":       ["Jan", "Jan", "Feb", "Jan", "Feb"],
    "amount":      [15000, 22000, 9000, 4300, 18000],
})

# --- Direct SUMIF/COUNTIF/AVERAGEIF equivalent: a boolean mask ---
mask = df["cost_center"] == "Marketing"
sumif_result   = df.loc[mask, "amount"].sum()
countif_result = mask.sum()
avgif_result   = df.loc[mask, "amount"].mean()

# --- SUMIFS equivalent: combine masks with & (AND) ---
mask2 = (df["cost_center"] == "Marketing") & (df["month"] == "Feb")
sumifs_result = df.loc[mask2, "amount"].sum()

# --- The scalable version: every combination at once, no repeated
    formulas ---
rollup = df.groupby(["cost_center", "month"])["amount"].agg(["sum", "
    count", "mean", "max"])
```

Listing 9: The native version: WHERE for a single SUMIF, GROUP BY for every combination at once

```
-- Direct SUMIFS equivalent: one specific combination
SELECT SUM(amount)
FROM transactions
WHERE cost_center = 'Marketing' AND month = 'Feb';

-- The scalable version: every cost_center x month combination in one
    query
SELECT
    cost_center,
    month,
    SUM(amount)   AS total,
    COUNT(*)      AS txn_count,
    AVG(amount)   AS avg_amount,
    MAX(amount)   AS max_amount
FROM transactions
GROUP BY cost_center, month
ORDER BY cost_center, month;
```

6 Error Handling

6.1 Reading Excel's Built-In Error Codes

Before suppressing an error, you need to know what it is actually telling you. Every Excel error code identifies a distinct category of problem:

Code	Typical cause	Typical fix
`#DIV/0!`	Division where the denominator is zero or blank	Guard the denominator, e.g. `IFERROR` or an explicit `IF(denom=0,...)` check
`#N/A`	A lookup (VLOOKUP/ MATCH/XLOOKUP) found no matching value	Confirm the lookup value actually exists and both sides match in type/spelling; wrap with `IFNA` if "not found" is a valid, expected outcome
`#NAME?`	Excel doesn't recognise text in the formula — usually a misspelled function name or a reference to a deleted Named Range	Re-check spelling; open Name Manager (Module 2) to see if a name was deleted
`#NULL!`	A range reference uses a space where a comma or colon was intended (Excel reads a space as "intersection of two ranges")	Fix the stray space/typo in the range reference
`#NUM!`	A numeric calculation is invalid (e.g. a negative number passed to `SQRT`, or an iterative function like `RATE` failing to converge)	Check the inputs feeding the function make mathematical sense
`#REF!`	A formula refers to a cell that no longer exists — most often because a referenced row/column/named-range source was deleted (Module 2, § Named Ranges)	Retrace what was deleted; re-point the formula or restore the reference
`#VALUE!`	A formula receives the wrong *type* of input — e.g. arithmetic on a cell that actually contains text	Check for numbers stored as text (Module 1/2) or a stray non-numeric character
`#SPILL!` (365)	A dynamic-array formula's results would overflow into cells that are already occupied	Clear the cells in the spill's path, or wrap in a function that trims the array
`#CALC!` (365)	A dynamic-array function returns something Excel's calc engine cannot represent (e.g. an empty array)	Adjust the formula logic so the array is never empty

6.2 IFERROR and IFNA

IFERROR

`=IFERROR(value, value_if_error)` — evaluates `value`; if it returns *any* error code (all nine above), returns `value_if_error` instead. Broad by design: it does not distinguish which error occurred.

IFNA

`=IFNA(value, value_if_na)` — narrower than `IFERROR`: catches *only* `#N/A`, letting every other error code (a genuine `#DIV/0!` or `#REF!` bug) surface visibly rather than being silently hidden.

IFERROR Can Hide a Real Bug, Not Just an Expected "No Result"

Wrapping an entire complex formula in `IFERROR(..., 0)` or `IFERROR(..., "")` is tempting because it makes error codes disappear from a report — but it also silently swallows a genuine `#REF!` from a broken reference or a `#DIV/0!` from a real logic bug, replacing a visible red flag with a quiet, wrong-looking zero. This is a common way subtly incorrect financial models pass a superficial visual review. The safer pattern is to wrap only the specific sub-expression expected to fail for a *known, acceptable* reason (e.g. only the denominator of a ratio, only a lookup that legitimately might not find a match), using `IFNA` instead of `IFERROR` whenever the only acceptable failure mode is "not found."

6.3 IS-Family Testing Functions

`ISERROR`, `ISNA`, `ISBLANK`, `ISNUMBER`, and `ISTEXT` each return a plain `TRUE`/`FALSE` test, useful when you need custom branching *based on* an error rather than just replacing it:

Listing 10: A custom error message using IF + ISERROR, versus the simpler IFERROR

```
=IF(ISERROR(C5/D5), "Check denominator", C5/D5)
=IFERROR(C5/D5, "Check denominator")    ' equivalent, more concise
```

Finance Application

`IFNA` wrapped around a VLOOKUP/XLOOKUP against a rate table is the standard pattern for a pricing or tax-bracket lookup where "value not found" should display a clean message (`"Rate not set"`) rather than a raw `#N/A` in a client-facing report — while a genuine `#REF!` elsewhere in the same model, from an accidentally-deleted row, still surfaces and gets caught during review.

6.4 Python and SQL Equivalents

Listing 11: try/except as the direct IFERROR equivalent, plus the vectorised pandas idiom

```
def safe_divide(numerator, denominator):
    try:
        return numerator / denominator
    except ZeroDivisionError:
        return "Check denominator"

# Vectorised equivalent over a whole column, avoiding a Python-level loop
import numpy as np
import pandas as pd
df = pd.DataFrame({"num": [10, 20, 30], "den": [2, 0, 5]})
df["ratio"] = np.where(df["den"] == 0, np.nan, df["num"] / df["den"])
# pandas also raises no error on 0-division for floats -- it returns inf
   /-inf/nan
# directly, which is arguably closer to Excel's #DIV/0! than a Python
   exception is
```

Listing 12: NULLIF and COALESCE: SQL's guard-the-denominator and default-value patterns

```sql
-- NULLIF turns a 0 denominator into NULL, which makes the division return NULL
-- instead of raising a divide-by-zero error -- the SQL equivalent of guarding first
SELECT
    numerator,
    denominator,
    numerator / NULLIF(denominator, 0) AS ratio
FROM ratio_inputs;

-- COALESCE is the IFERROR/IFNA equivalent: first non-NULL value in the list
SELECT COALESCE(numerator / NULLIF(denominator, 0), 0) AS safe_ratio
FROM ratio_inputs;
```

7 Lookup Functions: VLOOKUP, HLOOKUP, XLOOKUP, and INDEX/MATCH

Key Concept

Every function in this section solves the same problem: given a value you know (a lookup key), retrieve a related value you don't — from somewhere else in the workbook. This is the single most-used analytical operation in Excel, and it is Excel's row-by-row simulation of what a relational database does natively with a `JOIN` (see the SQL box at the end of this section).

7.1 VLOOKUP

VLOOKUP

`=VLOOKUP(lookup_value, table_array, col_index_num, [range_lookup])`

- `lookup_value` — the key you're searching for.
- `table_array` — the range to search, with the lookup key in its *first (leftmost) column*.
- `col_index_num` — which column of `table_array` to return, counted from 1 starting at the leftmost column of the range (not the worksheet's actual column letter).
- `range_lookup` — `FALSE`/`0` for an **exact match** (the overwhelming majority of finance use cases: looking up an account ID, an employee ID, a ticker); `TRUE`/`1` (or omitted) for an **approximate match**, which requires the first column of `table_array` to be sorted ascending and returns the largest value *less than or equal to* `lookup_value` — the correct mode for a tax-bracket or fee-tier lookup against sorted breakpoints.

The Silent Approximate-Match Trap

`range_lookup` defaults to `TRUE` if omitted entirely. Leaving it out on what should have been an exact-match lookup (an account ID, an invoice number) does not error — it silently returns the nearest *smaller* match in an unsorted or wrongly-sorted table, producing a plausible-looking but wrong value with zero warning. **Always type `FALSE` explicitly** unless you have deliberately chosen approximate-match for a genuinely sorted breakpoint table.

VLOOKUP Cannot Look Left

`table_array`'s lookup key must be in its *first* column, and `col_index_num` can only count *rightward* from there. If the value you actually want to retrieve sits in a column to the *left* of the lookup key in the source data, VLOOKUP simply cannot reach it — the table must be restructured, or §7.4 / §7.3 used instead. This is the single biggest structural limitation that motivated both INDEX/MATCH and XLOOKUP.

col_index_num Breaks When Columns Are Inserted or Deleted

Because `col_index_num` is a hardcoded integer, it does not automatically adjust when a column is inserted into or deleted from the middle of `table_array` — exactly the positional-fragility problem flagged as a general theme back in Module 1 and Module 2's preview of this module. Inserting a new column between the lookup key and the target column silently shifts what `col_index_num` actually points at, without any error.

Lock `table_array` Before You Fill Down

`table_array` should almost always be an absolute reference (`F4` once, Module 2, § Absolute vs. Relative References) before filling a VLOOKUP down a column — otherwise the lookup range itself drifts by one row for every row the formula is copied down, eventually searching a range that no longer contains the real table at all.

7.2 HLOOKUP

HLOOKUP

`=HLOOKUP(lookup_value, table_array, row_index_num, [range_lookup])` — the exact horizontal mirror of VLOOKUP: the lookup key sits in the *first row* of `table_array`, and `row_index_num` counts *downward* to the row being retrieved. Used when source data is laid out with categories running across columns rather than down rows (e.g. monthly figures as column headers) — comparatively rare in practice, since most tabular data (and every database export) is row-oriented, but it appears in transposed summary tables and pivoted time-series layouts.

7.3 XLOOKUP

XLOOKUP (Excel 2021/365 only) was built specifically to eliminate VLOOKUP's structural weaknesses.

XLOOKUP

`=XLOOKUP(lookup_value, lookup_array, return_array, [if_not_found], [match_mode], [search_mode])`

- `lookup_array` and `return_array` are **separate** ranges — the return column can be anywhere relative to the lookup column, including to its *left*, solving VLOOKUP's biggest limitation directly.
- `if_not_found` — an optional built-in replacement value for a failed match, making a separate `IFNA` wrapper unnecessary in most cases.
- `match_mode` — `0` = exact match (**the default**, notably the opposite default from VLOOKUP's approximate-match default), `-1` = exact match or next *smaller* item, `1` = exact match or next *larger* item, `2` = wildcard match.
- `search_mode` — controls search direction/algorithm (first-to-last, last-to-first, or binary search on sorted data); rarely changed from its default.

Listing 13: XLOOKUP solving VLOOKUP's left-lookup problem directly, with a built-in not-found message

```
=XLOOKUP(B5, AccountTable[AccountID], AccountTable[AccountName], "Account
    Not Found")
```

Key Concept

Because `lookup_array` and `return_array` are independently specified ranges rather than one `table_array` plus a column offset, inserting or deleting a column between them has *no effect at all* on an XLOOKUP formula — it is structurally immune to the exact fragility that breaks VLOOKUP's `col_index_num`.

XLOOKUP Is a Compatibility Trap in Shared or Legacy Files

XLOOKUP does not exist in Excel 2019 and earlier (non-365 licenses), and a workbook containing it will show `#NAME?` errors for anyone opening it in an older version. In any environment where the file might be opened outside Microsoft 365 — a common constraint at organisations still on perpetual-license Excel — INDEX/MATCH remains the safer, universally-compatible choice despite XLOOKUP's cleaner syntax.

7.4 INDEX and MATCH

Used together, `INDEX` and `MATCH` predate XLOOKUP by decades and solve the exact same left-lookup and column-insertion problems, at the cost of needing two nested functions instead of one.

INDEX and MATCH

`=INDEX(array, row_num, [col_num])` — returns the value at position (`row_num`, `col_num`) within `array`, counting from 1 at the array's own top-left corner.

`=MATCH(lookup_value, lookup_array, [match_type])` — returns the *position number* where `lookup_value` is found within `lookup_array` (not the value itself). `match_type`: `0` = exact match, `1` = largest value ≤ lookup (ascending-sorted array), `-1` = smallest value ≥ lookup (descending-sorted array).

Combined: `=INDEX(return_array, MATCH(lookup_value, lookup_array, 0))` — `MATCH` finds the row position, and `INDEX` retrieves the value at that position from a completely independent range, exactly like XLOOKUP's two-array design.

Listing 14: INDEX/MATCH performing the same left-lookup XLOOKUP did above

```
=INDEX(AccountTable[AccountName], MATCH(B5, AccountTable[AccountID], 0))
```

Two-Dimensional INDEX/MATCH: Looking Up by Row *and* Column

Nesting a second `MATCH` inside `INDEX`'s `col_num` argument performs a true two-way lookup — finding a value at the intersection of a row key *and* a column key, exactly the shape of a rate table or a two-variable sensitivity grid (§9.2):

```
=INDEX(RateGrid, MATCH(RowKey, RowHeaders, 0), MATCH(ColKey,
    ColHeaders, 0))
```

7.5 Choosing the Right Lookup Function

	VLOOKUP	HLOOKUP	XLOOKUP	INDEX/MATCH
Direction	Vertical only	Horizontal only	Either	Either
Can look left/up	No	No	Yes	Yes
Default match	Approximate (risky)	Approximate	Exact (safe)	Exact if `0` set
Survives column insert	No	No	Yes	Yes
Excel version	All	All	2021/365 only	All
Built-in not-found value	No	No	Yes	No (wrap in IFNA)

Finance Application

Approximate-match VLOOKUP against a sorted breakpoint table is the textbook way to implement a tax bracket or a fee-tier lookup (a small table of `[0, rate1], [threshold1, rate2], [threshold2, rate3]...`). Exact-match lookups (VLOOKUP with `FALSE`, or XLOOKUP/INDEX-MATCH) pull GL account descriptions from a chart-of-accounts table, pull a customer's credit rating by ID, or — with the two-dimensional INDEX/MATCH pattern above — retrieve a specific historical stock price given a date and a ticker from a matrix layout.

#N/A From "Identical-Looking" Values — Type and Whitespace Mismatches

An exact-match lookup that fails with `#N/A` despite the value visibly appearing in the table is almost always one of: (1) a type mismatch — the lookup key is text `"1024"` in one place and a genuine number `1024` in the other (a direct callback to the numbers-stored-as-text issue from Modules 1–2); (2) trailing/leading whitespace invisible to the eye but present in the underlying string; (3) a genuinely different Unicode character that renders identically (common when data is pasted from a PDF or web source). `TRIM()` (§8) and `VALUE()` are the standard first checks.

7.6 Python and SQL Equivalents

Key Concept

This is the section where the Python and SQL equivalents are not just "similar tools" but the *actual, native operations* that all four Excel lookup functions are manually simulating. A `VLOOKUP` is, function for function, one column of a `LEFT JOIN`.

Listing 15: pandas: .map() for a simple single-column lookup, merge() for the full VLOOKUP/INDEX-MATCH equivalent

```
import pandas as pd

accounts = pd.DataFrame({
    "AccountID": [101, 102, 103],
    "AccountName": ["Cash", "Accounts Receivable", "Inventory"],
})
transactions = pd.DataFrame({"AccountID": [102, 101, 103, 999]})

```

```python
# --- .map(): the simplest case, a dict-like single-column lookup ---
lookup_dict = accounts.set_index("AccountID")["AccountName"]
transactions["AccountName"] = transactions["AccountID"].map(lookup_dict)
# Unmatched keys (999) become NaN automatically -- pandas's version of #N/A

# --- merge(): the full VLOOKUP/XLOOKUP/INDEX-MATCH equivalent, join-based ---
result = transactions.merge(accounts, on="AccountID", how="left")
# how="left" keeps every transaction row even if no account match is found,
# exactly mirroring VLOOKUP's behaviour of returning #N/A rather than dropping the row
```

Listing 16: LEFT JOIN: the native, set-based version of every lookup function above, in one pass over the whole table

```sql
SELECT
    t.transaction_id,
    t.account_id,
    a.account_name          -- NULL here is SQL's #N/A: no match found
FROM transactions AS t
LEFT JOIN chart_of_accounts AS a
    ON t.account_id = a.account_id;
```

Why This Comparison Matters for a Tech-Background Learner

A `VLOOKUP` runs its search logic once *per formula, per row*, refetching and re-searching the table array every single time it's copied down a column of thousands of rows — `JOIN` performs the equivalent match for every row in one optimised, indexable, set-based pass. This is precisely why a spreadsheet with thousands of VLOOKUP formulas against a large table becomes noticeably slow (tying back to Module 2's calculation-mode discussion), while the equivalent SQL `JOIN` or pandas `merge()` over the same data volume is close to instantaneous — the lookup family is genuinely one of the strongest cases in this entire handbook for reaching for a database instead of a spreadsheet once row counts grow large.

8 String Functions

Real-world data is rarely clean. String functions turn messy, glued-together text into structured, usable values — as a *live formula*, unlike Text to Columns (Module 2), which only acts once on a fixed selection.

8.1 Extracting Substrings by Position

LEFT, RIGHT, MID

`=LEFT(text, [num_chars])` — the first `num_chars` characters.
`=RIGHT(text, [num_chars])` — the last `num_chars` characters.
`=MID(text, start_num, num_chars)` — `num_chars` characters starting at position `start_num`.

8.2 Finding a Position, and Extracting by It

FIND and SEARCH

`=FIND(find_text, within_text, [start_num])` — **case-sensitive**, no wildcards; returns the character position of the first match, or `#VALUE!` if not found.
`=SEARCH(find_text, within_text, [start_num])` — identical, but **case-insensitive** and **supports wildcards** (`*`, `?`).

Combining `FIND`/`SEARCH` with `LEFT`/`MID` extracts a variable-length substring around a delimiter — the exact live-formula version of what Text to Columns does once, statically:

Listing 17: Extracting the year and the sequence number from "INV-2026-00451" without Text to Columns

```
=MID(A2, FIND("-",A2)+1, 4)                          ' -> 2026
=MID(A2, FIND("-",A2,FIND("-",A2)+1)+1, 5)           ' -> 00451
```

Finance Application

This directly answers the weakness Module 2 identified in Text to Columns — that it is a one-time operation that doesn't cover rows added later. A formula built with `FIND`/`MID` re-parses every new invoice number automatically as rows are appended below, with no need to ever re-run a wizard.

8.3 Cleaning Text

TRIM and CLEAN

`=TRIM(text)` — removes leading/trailing spaces and collapses multiple internal spaces to one (does *not* remove tabs or line breaks). `=CLEAN(text)` — strips non-printable control characters, common in data pasted from a PDF or a legacy system export.

TRIM Is the First Fix for the #N/A Whitespace Trap

The invisible-whitespace lookup failure flagged in §7 is fixed precisely by wrapping the lookup key in `TRIM()` on both sides of the match — one of the most common single-line fixes in real financial-data cleanup work.

8.4 Case Conversion

`=UPPER(text)`, `=LOWER(text)`, `=PROPER(text)` (capitalises the first letter of each word) — used to standardise inconsistent capitalisation before an exact-match lookup or a duplicate check, since Excel's lookup functions are not case-sensitive by default but visually inconsistent casing still looks unprofessional in a report.

8.5 Joining Text

CONCATENATE, CONCAT, TEXTJOIN, and &

`=A1&" "&B1` — the `&` operator, the simplest way to join text, available in every version.
`=CONCATENATE(text1, text2, ...)` — the original function form; Microsoft has marked it legacy in favour of `CONCAT`.
`=CONCAT(text1, text2, ...)` — its modern replacement; also accepts ranges directly.
`=TEXTJOIN(delimiter, ignore_empty, text1, [text2], ...)` — joins a whole range with a chosen delimiter in one call, and can skip blank cells automatically — meaningfully more capable than the other three for joining a variable-length list (e.g. a comma-separated list of active product codes) without manually chaining dozens of `&` operators.

Finance Application

`TEXTJOIN` is the standard way to build a dynamic report title (`="Monthly P&L -- "&TEXT(EOMONTH(TODAY(),0),"mmmm yyyy")`, combining a text function with the `EOMONTH` date function from §10.2) so the title updates itself every month instead of being retyped.

8.6 Replacing Text: SUBSTITUTE vs. REPLACE

SUBSTITUTE and REPLACE

`=SUBSTITUTE(text, old_text, new_text, [instance_num])` — replaces based on *matching text content*; optionally target only the n-th occurrence.
`=REPLACE(old_text, start_num, num_chars, new_text)` — replaces based on *fixed character position*, regardless of what text is actually there.

Use `SUBSTITUTE` when you know *what* to replace (e.g. every hyphen with a space); use `REPLACE` when you know *where* to replace (e.g. always mask the first four digits of an account number, whatever those digits happen to be).

8.7 Number-Text Conversion: TEXT and VALUE

TEXT and VALUE

`=TEXT(value, format_text)` — converts a number/date into a formatted *text string*, using the same format-code syntax as the Number Format dialog (Module 1). Essential for embedding a formatted number inside a joined text string, since `&` otherwise displays the raw unformatted value.
`=VALUE(text)` — the reverse: converts numeric-looking text back into a genuine number, a formula-based alternative to Module 2's Text-to-Columns "silent fix" for numbers stored as text.

FIND/SEARCH Return #VALUE! When Nothing Is Found

Unlike a lookup function's `#N/A`, a failed `FIND` or `SEARCH` returns `#VALUE!` — easy to mis-diagnose as a type error elsewhere in the formula rather than "the delimiter wasn't present in this particular cell." Wrap in `IFERROR` (§6) whenever the delimiter isn't guaranteed to exist in every row.

8.8 Python and SQL Equivalents

Listing 18: String slicing and pandas' vectorised .str accessor

```python
s = "INV-2026-00451"
year = s[4:8]           # LEFT/MID equivalent via slicing
seq  = s[9:]             # RIGHT-style slice to the end

import pandas as pd
df = pd.DataFrame({"invoice": ["INV-2026-00451", "INV-2026-00452"]})
df["year"] = df["invoice"].str.split("-").str[1]         # split-based extraction
df["clean_name"] = df["invoice"].str.strip().str.upper()   # TRIM + UPPER equivalent
df["joined"] = df["invoice"].str.cat(df["year"], sep=" | ")   # TEXTJOIN equivalent
df["masked"] = df["invoice"].str.replace(r"^\w+", "XXX", regex=True)   # REPLACE-by-position idea
```

Listing 19: The direct SQL string-function equivalents

```sql
SELECT
    invoice_number,
    SUBSTRING(invoice_number FROM 5 FOR 4)      AS invoice_year,    -- MID equivalent
    TRIM(invoice_number)                        AS trimmed,
    UPPER(invoice_number)                       AS upper_form,
    CONCAT(invoice_number, ' - ', status)       AS joined,
    REPLACE(invoice_number, '-', ' ')           AS substitute_equiv
FROM invoices;
```

9 What-If Analysis

9.1 The Core Idea

Key Concept

Every formula you have built so far answers a *forward* question: given these inputs, compute this output. **What-If Analysis** is the family of tools built for the reverse and the exploratory questions that a single static formula cannot answer on its own: *"If this input changes, what happens to that output?"* and *"What input value do I need to hit a specific target output?"* Three built-in tools, all under **Data** tab → **Forecast** group → **What-If Analysis**, answer three distinct shapes of this question:

Tool	Question it answers	Direction
Data Table	"Show me the output across a whole range of one or two input values, all at once."	Forward: many inputs → many outputs, one formula
Scenario Manager	"Show me the output under a few named, curated combinations of several inputs at once."	Forward: several named input bundles → one output snapshot each
Goal Seek	"What single input value do I need, to make this output hit an exact target?"	Backward: one target output → one solved input

9.2 Data Tables

A **Data Table** recalculates a model's output formula repeatedly, once for every value in a test range, and lays the results out in a grid — without you writing a separate formula for each test value.

9.2.1 One-Variable Data Table

Building a One-Variable Data Table

1. List the test values for the one input you want to vary, down a column (or across a row).
2. In the cell diagonally adjacent to the top of that list (one row up, one column over), enter a formula that references the model's output cell — e.g. `=NetProfit`.
3. Select the entire block: the output-reference cell, the column of test values, and the empty column beside them where results will appear.
4. **Data → What-If Analysis → Data Table**. If your test values run *down a column*, enter the model's real input cell in **Column Input Cell** and leave **Row Input Cell** blank (and vice versa if your test values run across a row).

The Row/Column Input Cell Naming Is Backwards From What Most People Expect

The single most common Data Table mistake: the **Row Input Cell** box is for test values arranged *across a row* (i.e. one row of values, tested by substituting each one into a single input cell), and **Column Input Cell** is for test values arranged *down a column*. It is easy to instinctively reach for "Row Input Cell" because you're thinking about the model's input row, rather than about where your test values themselves are positioned — the box you need is determined entirely by the *orientation of your test-value list*, not by anything about the model.

9.2.2 Two-Variable Data Table

A two-variable table varies *two* inputs simultaneously, one down the rows and one across the columns, tracking a single output at every intersection.

Net Profit	Sales test values			
COGS	**280,000**	**295,000**	**310,000**	**325,000**
200,000	37,920	45,030	52,140	59,250
215,000	30,810	37,920	45,030	52,140
230,000	23,700	30,810	37,920	45,030
250,000	14,220	21,330	28,440	35,550

Figure 1: A two-variable Data Table: Sales test values run across the top row, COGS test values run down the left column, and every interior cell is Net Profit recalculated for that Sales/COGS combination. In Excel, the top-left corner cell contains a live reference such as `=NetProfit`, not a text label, and must sit exactly at the top-left of the selected block.

Building a Two-Variable Data Table

1. Top-left corner cell of the block: a formula referencing the model's output (here, `=NetProfit`) — *not* a text label.
2. Across the top row, to the right of the corner cell: the test values for the input that will vary by column (Sales, in Figure 1).
3. Down the left column, below the corner cell: the test values for the input that will vary by row (COGS, in Figure 1).
4. Select the *entire* block — corner cell, top row of values, left column of values, and every empty cell in between.
5. **Data → What-If Analysis → Data Table**. **Row Input Cell** = the model's real Sales input cell (because Sales values run *across the top row*); **Column Input Cell** = the model's real COGS input cell (because COGS values run *down the left column*).

Excel fills every interior cell with an array formula of the form `{=TABLE(row_input_cell, column_input_cell)}` — you cannot see or edit this text directly the way you edit a normal formula, and you cannot delete just one cell of the result grid; the whole table must be cleared together.

Key Concept

A Data Table is not a separate calculation engine — it silently re-runs the *entire real model*, once per grid cell, substituting each test-value pair into the actual input cells each time. This is exactly why a large two-variable Data Table (say, a 20×20 grid) can make a heavy model noticeably sluggish: Excel is recalculating the whole workbook 400 times in the background. This directly connects to the calculation-mode discussion in Module 2 (§ Excel Options): **Automatic Except for Data Tables** is a calculation mode built specifically to let everything else in the workbook recalculate live while Data Tables only refresh on `F9`, precisely to prevent this slowdown.

Finance Application

The Figure 1 example — Net Profit sensitivity to Sales and COGS — is the textbook two-way sensitivity table used throughout equity research and LBO models. Notice that this achieves the identical *visible result* as the manually-built, mixed-reference sensitivity grid constructed in Module 2's worked example (§ Worked Example, using `$B7` and `C$6`-style mixed references written by hand). The trade-off between the two approaches is real: a Data Table re-runs your *actual* model logic exactly as built (no risk of the grid's formula drifting out of sync with the real model), but produces an opaque array formula that is harder to audit cell-by-cell; a manually-built mixed-reference grid is fully auditable and portable to another sheet, but requires you to correctly re-derive the model's formula inside the grid itself, with a real risk of that copy silently diverging from the real model over time.

The Corner Cell Must Be a Formula, Not a Label

Typing a text label like `"Net Profit Sensitivity"` into the corner cell instead of an actual formula reference (`=NetProfit`) is a common early mistake — Data Table needs a genuine live reference there to know what output to track through every recalculation.

9.3 Scenario Manager

Where a Data Table exhaustively tests every combination across a continuous range of one or two inputs, **Scenario Manager** does the opposite: it lets *many* inputs change together, but only for a small, deliberately curated, named set of combinations — "Best Case," "Base Case," "Worst Case," rather than every possible value.

Creating and Adding Scenarios

1. **Data → What-If Analysis → Scenario Manager → Add**.
2. **Scenario name** — a descriptive label (e.g. `"Best Case"`).
3. **Changing cells** — select every input cell this scenario should control; hold `Ctrl` to select multiple non-adjacent cells (e.g. a growth-rate assumption *and* a margin assumption *and* an interest-rate assumption, all in one scenario).
4. **Comment** — auto-populated with your name and the creation date; editable.
5. Click **OK**, then enter the specific value each changing cell should take *under this scenario* in the next dialog.
6. To **add more scenarios**, click **Add** again from the main Scenario Manager dialog and repeat — each new scenario reuses the *same* set of changing cells (Excel remembers them after the first scenario is created) but stores a different value for each.

Selecting a scenario from the list and clicking **Show** instantly overwrites every changing cell in the live worksheet with that scenario's stored values, letting you watch every dependent formula — including charts — update in real time.

9.3.1 Analysing Multiple Scenarios at Once: The Scenario Summary Report

Clicking **Show** one scenario at a time is fine for spot-checking, but comparing every scenario side by side needs the **Summary** button inside the Scenario Manager dialog.

	Current	Best Case	Base Case	Worst Case
Changing Cells				
Sales	300,000	340,000	300,000	260,000
COGS	210,000	220,000	210,000	205,000
Margin	20%	24%	20%	15%
Result Cells				
Net Profit	14,220	22,752	14,220	6,517

Figure 2: A Scenario Summary report, generated automatically on a new worksheet. **Changing Cells** (here `$C$4` and `$C$3`) are shown first, shaded grey, with each scenario's stored input values in its own column; **Result Cells** (`$C$13`, the tracked output) follow below, showing what that output computes to under each scenario. The "Current Values" column is a frozen snapshot of the changing cells' values *at the moment the report was generated* — it does not represent any scenario you saved.

Generating a Scenario Summary

Scenario Manager → **Summary** → choose **Scenario Summary** (a static outlined worksheet, as in Figure 2) or **Scenario PivotTable** (the same comparison, but pivotable/filterable like a normal PivotTable) → select the **Result cells** you want tracked (the output cell(s) whose value should be shown for every scenario — `$C$13` in the figure).

Key Concept

This is Scenario Manager's core advantage over a Data Table: it can vary **several inputs simultaneously as one named bundle** (a growth rate *and* a margin *and* a tax rate together, all shifting at once for "Recession"), which a Data Table — capped at two varying inputs — structurally cannot do. The cost is coverage: Scenario Manager only shows you the handful of specific combinations you explicitly defined, not every possible combination across a continuous range.

The Scenario Summary Report Is a Static Snapshot, Not a Live View

The worksheet generated by **Summary** is a frozen picture of the scenarios *as they existed at the moment you clicked Summary*. If you later edit a scenario's stored values (Scenario Manager → **Edit**), the existing Scenario Summary sheet does *not* update — you must regenerate it. This is the opposite behaviour from a Data Table, which is always live. Always re-run Summary after editing any scenario before sharing a report built from it.

Changing Cells Should Be Genuine Inputs, Not Formulas

Scenario Manager works by literally overwriting the stored value into each changing cell when you click **Show** — it is designed for pure input/assumption cells, not formula cells. Selecting a formula cell as a "changing cell" means Show will overwrite that formula with a hardcoded number, silently breaking it.

Finance Application

Bull/Base/Bear equity-research cases, each bundling a revenue-growth assumption, a margin assumption, and a discount-rate assumption together under one named scenario, are the canonical Scenario Manager use case — precisely matching Figure 2's pattern of naming and comparing distinct `Net profit` outcomes (labelled `Net profit` and `NP2` in the figure) side by side against a shared set of changing cells.

9.4 Goal Seek

Goal Seek reverses the usual direction of a formula: instead of computing an output from inputs, you specify the *output you want* and Goal Seek iteratively solves backward for the single input value that produces it.

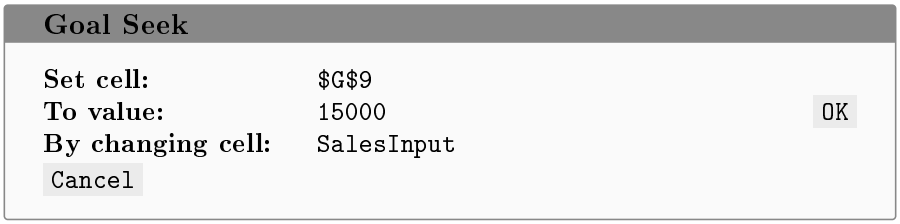


Figure 3: The Goal Seek dialog, mid-setup on an income-statement model. **Set cell** (`$G$9`) must contain a formula — here, one of the income statement's subtotal cells. **To value** (`15000`) is the exact numeric target you want that formula to equal. **By changing cell** is left to be filled with the single input cell Goal Seek is allowed to adjust (typically an upstream driver such as Sales) to make `$G$9` hit `15000`.

Using Goal Seek

1. **Data → What-If Analysis → Goal Seek**.
2. **Set cell** — the cell containing the formula/output you want to hit a target (must be a formula, not a hardcoded value).
3. **To value** — type the exact target number, hardcoded (not a cell reference).
4. **By changing cell** — the *single* input cell Goal Seek is allowed to vary; this cell must feed into **Set cell**'s formula chain, directly or indirectly.
5. Click **OK**. Excel runs an internal iterative search (conceptually similar to a numerical root-finding method) and reports a **Goal Seek Status** dialog.

9.4.1 Reading the Goal Seek Status Dialog

After solving, the status dialog reports either *"Goal Seeking with Cell [X] found a solution"* or *"Goal Seeking with Cell [X] may not have found a solution"*, alongside the **Target value** you asked for and the **Current value** it actually reached:

- If **Target value** and **Current value** match (or are within Excel's internal convergence tolerance — by default, up to 100 iterations or until the change between iterations falls below 0.001), the solve succeeded, and clicking **OK** commits the found value permanently into the changing cell.
- If Excel reports it *may not have* found a solution, the relationship between the changing cell and the target either isn't smooth/monotonic enough for the iterative search to converge, or the target is genuinely unreachable within the model's logic (e.g. asking for a Sales figure that would require a negative COGS elsewhere) — in this state, do not trust the reported "Current value"; treat it as a failed search rather than an approximate answer.

Goal Seek Has No Concept of Constraints

Goal Seek will happily report a "solution" that is a negative Sales figure, an interest rate above 100%, or any other value that is mathematically consistent with the formula but practically nonsensical — it has no built-in bounds or plausibility checks. Always sanity-check the solved value, not just whether Status reports success. The **Solver** add-in (not covered in this module, but worth knowing by name) extends this exact idea with actual constraints and multiple simultaneous changing cells — the natural next tool once Goal Seek's one-input, one-target, no-constraints design becomes limiting.

Goal Seek Permanently Overwrites the Changing Cell

Clicking **OK** on the Status dialog writes the solved value directly into **By changing cell**, overwriting whatever was there — including a real planning assumption you may still need elsewhere. If you only want to *see* whether a target is reachable without committing the change, click **Cancel** instead of **OK**; the sheet reverts to its pre-Goal-Seek state untouched. If you did click OK and need the original value back, `Ctrl+Z` immediately undoes it as a single step.

Finance Application

Classic Goal Seek questions: "What Sales figure sets Net Profit exactly to break-even (zero)?"; "What discount rate makes an investment's NPV exactly zero?" (manually deriving what a dedicated `IRR` function, covered in a later module, computes directly); "What Sales growth rate is required to satisfy an EBITDA-margin debt covenant?" Figure 3's example — solving for whatever upstream driver makes a subtotal hit `15000` — is exactly this pattern.

9.5 Choosing the Right What-If Tool

	Goal Seek	1-Var Table	2-Var Table	Scenario Mgr
Question direction	Backward	Forward	Forward	Forward
Inputs varied	1	1	2	Any number
Outputs tracked	1	Any number	1	Any number
Coverage	1 exact target	Full range	Full grid	A few named bundles
Result form	Single value	Column/row	Grid	Comparison report
Live or static	Live cell change	Live grid	Live grid	Static report

9.6 Python and SQL Equivalents

Key Concept

This is the one place in the handbook so far where the three-way translation genuinely breaks down for one of the tools — and it is worth stating honestly rather than forcing a strained equivalent. Data Tables and Scenario Manager translate naturally into Python (and, with real effort, into SQL); Goal Seek's backward, iterative-solving nature has no clean SQL equivalent at all, because SQL is not an iterative numerical solver.

Listing 20: All three What-If tools in Python: this is dramatically less friction than in Excel, because a Python function already takes parameters

```
import itertools
import numpy as np
import pandas as pd
from scipy.optimize import brentq

def net_profit(sales, cogs, tax_rate=0.21):
    gross_profit = sales - cogs
    ebt = gross_profit * 0.6          # simplified: SG&A/EBITDA/D&A/
        Interest folded in
    return ebt * (1 - tax_rate)

# --- Two-Variable Data Table equivalent: itertools.product + a
    comprehension ---
sales_values = np.arange(280000, 330000, 5000)
cogs_values  = np.arange(200000, 255000, 5000)
grid = pd.DataFrame(
    [[net_profit(s, c) for s in sales_values] for c in cogs_values],
    index=cogs_values, columns=sales_values
)  # a Data Table is really just a nested loop that Excel hides behind a
    dialog

# --- Scenario Manager equivalent: a dict of named input bundles ---
scenarios = {
    "Best Case":  {"sales": 325000, "cogs": 200000},
    "Base Case":  {"sales": 291550.5, "cogs": 228429.8},
    "Worst Case": {"sales": 280000, "cogs": 250000},
}
summary = pd.Series({name: net_profit(**inputs) for name, inputs in
```

```
    scenarios.items()})

# --- Goal Seek equivalent: scipy's root-finder solves backward for Sales ---
target_profit = 15000
solved_sales = brentq(lambda s: net_profit(s, 228429.8) - target_profit,
                      a=0, b=10_000_000)
```

Why What-If Analysis Feels Clunky in Excel but Trivial in Python

A Python function inherently *takes parameters* — calling `net_profit(325000, 200000)` a hundred times with different arguments is just a loop. An Excel formula, by contrast, is permanently wired to specific cell addresses; there is no native way to "call the Net Profit formula with different inputs" without physically changing what's in those cells first. Goal Seek, Data Tables, and Scenario Manager exist *specifically* to work around that structural limitation — they are Excel's compensating mechanisms for not having first-class functions with parameters, which is precisely why the equivalent operations require no special named tool at all in Python.

Listing 21: Data Table and Scenario Manager translate into SQL reasonably well via CROSS JOIN; Goal Seek genuinely does not

```
-- Two-Variable Data Table equivalent: CROSS JOIN two input-value tables,
-- the same pattern used in Module 2's worked-example sensitivity grid
SELECT
    sv.sales_test, cv.cogs_test,
    (sv.sales_test - cv.cogs_test) * 0.6 * (1 - 0.21) AS net_profit
FROM sales_test_values AS sv
CROSS JOIN cogs_test_values AS cv;

-- Scenario Manager equivalent: a table of named scenarios, one row each
SELECT scenario_name,
       (sales - cogs) * 0.6 * (1 - 0.21) AS net_profit
FROM scenario_assumptions;

-- Goal Seek: no honest native equivalent. Standard SQL has no iterative numerical
-- solver; simulating one requires either a recursive CTE that manually re-implements
-- an approximation algorithm, or -- more realistically -- pulling the data out to
-- Python/scipy as above. This is a genuine capability gap, not a missing keyword.
```

10 Core Financial Functions: PMT and Date Functions

10.1 The PMT Function

PMT

`=PMT(rate, nper, pv, [fv], [type])`

- `rate` — the **periodic** interest rate — must be expressed in the *same period* as `nper` (a monthly `nper` needs a monthly `rate`, i.e. the annual rate divided by 12).
- `nper` — the total number of payment periods over the life of the loan, in that same period unit.
- `pv` — present value, the loan principal (or investment amount).
- `fv` — future value (optional, defaults to `0` — a loan fully paid off by the end of `nper`).
- `type` — `0` (default) for payments due at the *end* of each period (an ordinary annuity — the standard case for a loan), `1` for payments due at the *start* of each period (an annuity due — typical for a lease).

Input / calculation	Value
Loan principal	500,000
Annual rate	10%
Term	5 years
Monthly rate	`=10%/12`
Payment periods	`=5*12`
Monthly payment	**10,623.52**

Figure 4: A worked PMT example: a Rs. 500,000 loan at a 10% annual rate over 5 years, paid monthly. The annual rate and year count are converted to a monthly rate (`r = 0.0083333`) and 60 payment periods before `=PMT(C7,C8,-500000)` returns Rs. 10,623.52 per month.

The Golden Rule of PMT — Match the Period

The single most common PMT error is mixing an annual rate with a monthly `nper` (or vice versa). Figure 4's example shows the correct conversion pattern explicitly, in two separate helper cells (`C7` = `rate/12`, `C8` = `years*12`) rather than trying to do the conversion inline inside the PMT call — a strongly recommended habit, since it makes the period-matching step visible and auditable rather than buried inside a single dense formula.

The Sign Convention

Excel's financial functions follow a strict **cash-flow sign convention**: money flowing *out* of you is negative, money flowing *in* is positive. In Figure 4, `pv` is entered as `-500000` (money you *received* as the loan — but written negative here specifically to make `PMT` return a *positive* payment figure). Enter `pv` as positive instead, and `PMT` returns a negative number, representing the same payment as an outflow — both are correct; the sign is a display choice, not a different calculation. What breaks the calculation is not choosing a sign convention, but being *inconsistent* within it.

Finance Application

PMT is the standard way to compute an EMI (equated monthly installment) for a mortgage, an auto loan, or a business term loan — exactly as shown in Figure 4. Run in reverse (with `pv` set to `0` and `fv` set to a savings target instead), the same function computes the monthly contribution required to reach a future savings goal.

The PMT Family — One Function per Unknown

PMT is one member of a small family of time-value-of-money functions that all share the same `rate`/`nper`/`pv`/`fv`/`type` arguments and the same sign convention, differing only in which one variable each solves for:

- `IPMT(rate, per, nper, pv, [fv], [type])` — the *interest* portion of a specific payment number (`per`).
- `PPMT(rate, per, nper, pv, [fv], [type])` — the *principal* portion of that same payment; `IPMT + PPMT` for any given period always equals the total `PMT`.
- `NPER(rate, pmt, pv, [fv], [type])` — solves for the number of periods, given a fixed payment.
- `RATE(nper, pmt, pv, [fv], [type])` — solves for the periodic interest rate, given a fixed payment.
- `PV` / `FV` — solve for present or future value directly.

Before knowing any of these exist, Goal Seek (§9.4) is exactly how you would manually back into, say, an implied interest rate: set `Set cell` to a PMT-formula output, `To value` to the known payment, and `By changing cell` to the rate assumption — Goal Seek and `RATE()` solve the identical problem, one iteratively and the other analytically.

A Correct-Looking but Flipped-Sign Result

Getting a PMT result with the "wrong" sign (negative when a positive was expected, or vice versa) is not a calculation error — it means `pv` was entered with the opposite sign from what was intended. The *magnitude* is still correct; only the sign convention needs adjusting, either by negating `pv` or by wrapping the whole call in a unary minus.

10.2 Date Functions

Formula	Purpose	Example result
`=TODAY()`	Current date	15-Aug-2026
`=TODAY()+10`	Ten days ahead	25-Aug-2026
`=EOMONTH(TODAY(),0)`	Current month-end	31-Aug-2026
`=DATEDIF(A2,TODAY(),"Y")`	Complete years elapsed	36
`=YEAR(A2)` / `=MONTH(A2)`	Date components	2026 / 8
`=DATE(2026,8,31)`	Construct a date	31-Aug-2026

Figure 5: Core date functions and their outputs: `TODAY()` for today's date, date arithmetic (`TODAY()+10`) for a future date, `EOMONTH()` for a month-end date, `DATEDIF()` for elapsed time (e.g. age from a birthdate), and `YEAR()`/`MONTH()`/`DAY()`/`DATE()` for extracting and reconstructing date components.

Dates Are Just Numbers

Underneath the display formatting, every Excel date is stored as a **serial number** — an integer count of days since a fixed reference point (day 1 = 1 January 1900, on Windows Excel). This is precisely why `TODAY()+10` in Figure 5 works as plain addition: it is genuinely adding 10 to an integer, which Excel then re-displays using the cell's date format. It is also why two dates can be subtracted directly (`EndDate - StartDate`) to get a plain number of days.

TODAY and NOW

`=TODAY()` — the current date, no time component, refreshed every time the workbook recalculates. `=NOW()` — current date *and* time, same refresh behaviour. Both take no arguments and are **volatile** — exactly the category of function flagged in Module 2's calculation-mode discussion as a common cause of a large workbook recalculating (and slowing down) more often than necessary.

EOMONTH

`=EOMONTH(start_date, months)` — the last day of the month, `months` away from `start_date`. `months=0` gives the end of the *current* month; negative values step backward, positive values step forward. `=EOMONTH(TODAY(),0)` is the standard formula for "the end of this reporting period."

DATEDIF

`=DATEDIF(start_date, end_date, unit)` — elapsed time between two dates. `unit`: `"Y"` (complete years), `"M"` (complete months), `"D"` (total days), `"YM"` (remaining months after whole years), `"MD"` (remaining days after whole months), `"YD"` (remaining days after whole years). Notably absent from the Formula AutoComplete dropdown and from Excel's own function documentation for years — it still works perfectly when typed manually, and remains the standard way to compute an age or a tenure.

YEAR, MONTH, DAY, and DATE

`=YEAR(date)`, `=MONTH(date)`, `=DAY(date)` — extract the corresponding numeric component from a date. `=DATE(year, month, day)` — the reverse: builds a genuine date value from three separate numbers. Combining extraction and reconstruction is the standard pattern for shifting a date by whole months while keeping it well-defined, e.g. `=DATE(YEAR(A2), MONTH(A2)+1, DAY(A2))` to roll a date forward exactly one month.

Two More Date Functions Worth Knowing, Even Though Not Pictured Above

`EDATE(start_date, months)` is EOMONTH's close cousin — it also shifts by whole months, but keeps the *same day-of-month* instead of jumping to month-end (the two are frequently confused: use EDATE when you want "the same day next month," EOMONTH when you specifically want "the last day of that month"). `WORKDAY(start_date, days, [holidays])` and `NETWORKDAYS(start_date, end_date, [holidays])` are business-day-aware — they skip weekends (and an optional supplied holiday list) automatically, and are the standard tools for a trade-settlement date (e.g. T+2) or a business-day-based deadline calculation.

Finance Application

`EOMONTH` generates correctly-spaced period-end dates for a loan amortisation schedule or a set of financial-statement column headers without manually typing each date. `DATEDIF` computes exact tenure for a depreciation schedule (age of an asset in years and months) or an employee's length of service. `DATE`/`YEAR`/`MONTH` together build a rolling trailing-12-month report header that updates itself every period instead of being retyped by hand — directly reusable inside the `TEXTJOIN` report-title pattern from §8.

Regional Date Formats Silently Misread on Import

A date typed or imported as `"03/04/2026"` means 3 April in a DD/MM/YYYY locale and 4 March in an MM/DD/YYYY locale — Excel applies whatever its regional setting expects, silently, with no warning that the *other* interpretation was possible. This is a classic and genuinely dangerous CSV-import problem, since a wrongly-parsed date often still looks like a perfectly plausible date. It directly connects back to Module 2's Text to Columns **Date** column-format option (which lets you specify the source's actual date order explicitly during import, rather than trusting Excel's regional guess), and to `DATEDIF`'s well-documented rounding quirks on its `"MD"` unit around certain month-boundary edge cases — a known Microsoft limitation worth being aware of, not a sign you've made an error.

10.3 Python and SQL Equivalents

Listing 22: PMT via numpy_financial (NumPy itself dropped its financial functions), and the date-function family via pandas/datetime

```python
import numpy_financial as npf   # pip install numpy-financial --break-system-packages
import pandas as pd
from dateutil.relativedelta import relativedelta
from datetime import date

# --- PMT: same sign convention as Excel (negative pv -> positive payment) ---
monthly_rate = 0.10 / 12
n_months     = 5 * 12
payment      = npf.pmt(monthly_rate, n_months, -500000)    # 10623.52...

# --- Date functions ---
today           = pd.Timestamp.today().normalize()           # TODAY()
future_date     = today + pd.Timedelta(days=10)               # TODAY()+10
end_of_month    = today + pd.offsets.MonthEnd(0)              # EOMONTH(TODAY(),0)
next_month_same_day = today + relativedelta(months=1)         # EDATE(TODAY(),1)

birthdate = pd.Timestamp("2000-01-01")
age_years = relativedelta(today, birthdate).years             # DATEDIF(...,"Y")

y, m, d = today.year, today.month, today.day                  # YEAR/MONTH/DAY
reconstructed = pd.Timestamp(year=y, month=m, day=d)          # DATE(y,m,d)
```

Listing 23: Date functions across common SQL dialects, and the honest gap for PMT

```sql
-- Portable across most dialects (syntax varies slightly by vendor):
SELECT
    CURRENT_DATE                                    AS today_equiv,        -- TODAY()
    CURRENT_DATE + INTERVAL '10 day'                AS plus_ten_days,      -- TODAY()+10
    DATEADD(month, DATEDIFF(month, 0, CURRENT_DATE) + 1, -1) AS eom,       -- EOMONTH (SQL Server idiom)
    DATEDIFF(year, birth_date, CURRENT_DATE)        AS age_years_approx,   -- DATEDIF(...,"Y")
```

```
    YEAR(CURRENT_DATE), MONTH(CURRENT_DATE), DAY(CURRENT_DATE)
                    -- YEAR/MONTH/DAY
FROM employees;

-- PMT: standard SQL has no built-in time-value-of-money functions at all.
-- This is a genuine, honest gap -- financial calculations of this kind are
-- normally done in the application layer (Python, Excel) and only the
-- resulting numbers are stored/queried in the database.
```

11 Worked Example: A Loan Sensitivity Model, Three Ways

This closing example combines nearly everything in this module into one small, realistic scenario: computing a loan's monthly payment, looking up a customer's approved interest rate by credit tier, guarding against a missing tier with error handling, building a two-way sensitivity table around the payment, and using Goal Seek to find the maximum loan a target payment can support.

11.1 Excel Implementation

1. **Inputs** (Named Ranges, Module 2): `LoanAmount` (`B2` = `500000`), `AnnualRate` (`B3`, looked up — see next point), `TermYears` (`B4` = `5`), `CreditTier` (`B5`, a Data-Validation dropdown sourced from a named list, Module 2).
2. **Rate lookup with error handling**: `B3` uses

   ```
   =IFNA(XLOOKUP(B5, RateTable[Tier], RateTable[Rate]), "Tier not found")
   ```

   This combines §7.3 and §6.
3. **PMT calculation**, period-matched explicitly per §10.1: `B7` = `=B3/12` (monthly rate), `B8` = `=B4*12` (months), `B10` = `=PMT(B7,B8,-LoanAmount)`.
4. **Two-variable Data Table** (§9.2): corner cell references `=B10`; loan amounts (Rs. 400,000–Rs. 600,000) run across the top row, annual rates (8%–14%) run down the left column; **Row Input Cell** = `LoanAmount`, **Column Input Cell** = the rate's source cell.
5. **Goal Seek** (§9.4): **Set cell** = `B10`, **To value** = the maximum affordable monthly payment (e.g. `12000`), **By changing cell** = `LoanAmount` — answering "what is the largest loan this payment can support, at the current rate?"

11.2 Python Implementation

Listing 24: The same model end-to-end: lookup, PMT, a sensitivity grid, and a Goal-Seek-style solve

```python
import numpy_financial as npf
import numpy as np
import pandas as pd
from scipy.optimize import brentq

rate_table = pd.DataFrame({"tier": ["A", "B", "C"], "rate": [0.08, 0.10, 0.13]})

def monthly_payment(loan_amount, annual_rate, term_years=5):
    return -npf.pmt(annual_rate / 12, term_years * 12, loan_amount)

# --- Rate lookup with error handling (XLOOKUP + IFNA equivalent) ---
```

```
credit_tier = "B"
matched = rate_table.loc[rate_table["tier"] == credit_tier, "rate"]
annual_rate = matched.iloc[0] if not matched.empty else None
if annual_rate is None:
    raise ValueError("Tier not found")

payment = monthly_payment(500_000, annual_rate)

# --- Two-Variable Data Table equivalent ---
loan_values = np.arange(400_000, 620_000, 20_000)
rate_values = np.arange(0.08, 0.145, 0.01)
grid = pd.DataFrame(
    [[monthly_payment(l, r) for l in loan_values] for r in rate_values],
    index=np.round(rate_values, 3), columns=loan_values
)

# --- Goal Seek equivalent: max loan supportable by a target payment ---
target_payment = 12000
max_loan = brentq(lambda l: monthly_payment(l, annual_rate) -
   target_payment,
                  a=0, b=5_000_000)
```

11.3 SQL Implementation

Listing 25: The lookup and the sensitivity grid, expressed as governed tables and a CROSS JOIN

```
CREATE TABLE rate_table (
    tier VARCHAR(5) PRIMARY KEY,
    annual_rate DECIMAL(5,4) NOT NULL
);
INSERT INTO rate_table VALUES ('A', 0.08), ('B', 0.10), ('C', 0.13);

CREATE TABLE loan_test_values  (loan_amount DECIMAL(12,2));
CREATE TABLE rate_test_values  (annual_rate DECIMAL(5,4));
-- populate loan_test_values with 400000..600000 step 20000,
-- and rate_test_values with 0.08..0.14 step 0.01 (omitted here for
   brevity)

-- The rate lookup: exactly a LEFT JOIN, VLOOKUP/XLOOKUP's native
   equivalent
SELECT c.customer_id, c.credit_tier, r.annual_rate
FROM customers AS c
LEFT JOIN rate_table AS r ON c.credit_tier = r.tier;

-- The two-variable sensitivity grid: CROSS JOIN, same pattern as the
   Data Table
-- equivalent earlier in this module and Module 2's worked example
SELECT
    lv.loan_amount, rv.annual_rate,
    -- Standard SQL has no PMT() -- the payment formula itself must be
       spelled out
    (lv.loan_amount * (rv.annual_rate/12)) /
        (1 - POWER(1 + rv.annual_rate/12, -60)) AS monthly_payment
FROM loan_test_values AS lv
CROSS JOIN rate_test_values AS rv;
-- Goal Seek's backward solve, again, has no native SQL equivalent.
```

12 Common Pitfalls & Debugging Checklist

Symptom	Likely cause & fix
A CF rule highlights the wrong cells entirely	The formula-based rule was written relative to the wrong anchor cell (§3.2) — rewrite it relative to the top-left cell of the actual selected range, and check **Applies to** in Manage Rules.
A nested-IF or IFS formula returns a plausible but wrong tier	A broader condition sits before a narrower one and shadows it (§4) — reorder from most specific/narrow to most general, or switch to `IFS` with explicit tiers.
SUMIFS returns `#VALUE!` or a wildly wrong total	Argument order reversed with SUMIF (sum range comes first in SUMIFS, last in SUMIF), or `sum_range`/`criteria_range` are different sizes (§5).
A formula shows `#REF!` after editing the sheet	A cell/row/column/named range the formula depended on was deleted (§6, Module 2) — open Name Manager or trace precedents to find and re-point the broken reference.
IFERROR is hiding a result that looks suspiciously clean (a flat 0 or blank)	The wrapper is masking a genuine bug, not just an expected edge case (§6) — narrow the wrap to only the sub-expression expected to fail, or switch to `IFNA` if "not found" is the only acceptable failure.
A VLOOKUP returns a value that's close but not exactly right	`range_lookup` defaulted to approximate match (TRUE) on data that needed exact match (§7.1) — add `FALSE` explicitly.
A VLOOKUP/XLOOKUP returns `#N/A` despite the value clearly being in the table	Type mismatch (text vs. number) or invisible whitespace in one side of the match (§7) — wrap both sides in `TRIM()`, or check with `ISNUMBER`/`ISTEXT`.
A lookup formula breaks after inserting a new column in the source table	`col_index_num` in a VLOOKUP is a hardcoded position that didn't shift with the insert (§7.1) — switch to XLOOKUP or INDEX/MATCH, both immune to this.
A two-variable Data Table shows the same value repeated in every cell, or an error across the whole grid	Row Input Cell and Column Input Cell were swapped, or the corner cell is a text label instead of a live formula reference (§9.2).
A Scenario Summary report doesn't reflect a scenario you just edited	The Summary report is a static snapshot generated once (§9.3) — regenerate it after any scenario edit.
Goal Seek reports it "may not have found a solution"	The target is genuinely unreachable given the model's logic, or the relationship between changing cell and target isn't smooth enough to converge (§9.4) — sanity-check whether the target is mathematically possible at all.
A PMT result is off by roughly a factor of 12 (or is a wildly wrong sign)	`rate` and `nper` are in mismatched periods (annual rate with monthly nper), or `pv`'s sign wasn't set deliberately (§10.1).
A date imported from CSV looks plausible but is actually the wrong day	Regional date-order mismatch (DD/MM vs. MM/DD) silently misread on import (§10.2) — re-import using Text to Columns' explicit Date column-format option (Module 2).

13 Chapter Summary & Self-Check

Before moving to Module 4, you should be able to, without looking anything up:

- ☐ Use Find and Replace's Options (wildcards, Match entire cell contents, Look in Formulas vs. Values, format-based search) to answer an audit question, not just a simple text swap.
- ☐ Build a formula-based Conditional Formatting rule, explain the top-left-cell anchoring convention, and use Stop If True correctly when rules conflict.
- ☐ Write a nested-IF chain and the equivalent IFS formula for the same three-or-more-tier decision, and explain why condition order matters in both.
- ☐ Correctly order the arguments of SUMIF vs. SUMIFS from memory, without checking the tooltip.
- ☐ Read every one of Excel's nine built-in error codes and state its most likely cause, and choose correctly between IFERROR and IFNA for a given situation.
- ☐ Explain why VLOOKUP cannot look left, and fix that limitation using either XLOOKUP or INDEX/MATCH.
- ☐ Build a two-dimensional INDEX/MATCH lookup, retrieving a value at the intersection of a row key and a column key.
- ☐ Use FIND/MID or SEARCH to extract a substring from delimited text as a live formula, and explain why this is more robust than a one-time Text to Columns split.
- ☐ Build both a one-variable and a two-variable Data Table, correctly distinguishing Row Input Cell from Column Input Cell.
- ☐ Create multiple Scenario Manager scenarios sharing the same changing cells, and generate (and correctly interpret) a Scenario Summary report.
- ☐ Use Goal Seek, correctly read its Status dialog, and explain its single-input/single-target limitation versus Scenario Manager and Data Tables.
- ☐ Build a PMT formula with correctly period-matched rate and nper, and explain the cash-flow sign convention.
- ☐ Use TODAY, EOMONTH, DATEDIF, YEAR/MONTH/DAY, and DATE together to build a self-updating, formula-driven date label.
- ☐ For each major topic in this module, name its closest Python and SQL equivalent — and state plainly, for What-If Analysis's Goal Seek, where that equivalence genuinely breaks down.

14 What's Next: Module 4 Preview

This module completed the "Excel Like a Pro" skill set — decision logic, conditional aggregation, robust lookups, error handling, live text manipulation, systematic what-if analysis, and the first two core financial functions. Two directions naturally continue from here, matching the original course's own structure:

- **Accounting Fundamentals & Financial Statements** (the original course's Modules 7–9) — what accounting actually is and why double-entry bookkeeping works the way it does, the accounting equation, the rules governing debits and credits, and then building the Income Statement, Balance Sheet, and Cash Flow Statement from scratch — the direct next step toward the three-statement modelling and DCF work later in the course.

- **Charts and PivotTables** (the original course's Module 10) — turning the SUMIFS-style manual conditional aggregation from this module into automatic, drag-and-drop summarisation, plus the visualisation tools (column/bar/line charts, slicers, timelines) that sit on top of it.

Since your course outline lists Accounting Fundamentals immediately after Financial Functions, Module 4 defaults to that track unless you'd rather pull the Charts/PivotTables thread first — let me know which, or send over your notes/screenshots from whichever lecture you'd like covered next, the same way you did for this module's What-If Analysis and PMT/date content.